

BAHRIA UNIVERSITY ISLAMABAD

Department of Computer Science

Faculty of Computing and Information Technology

Semester Project Proposal

SkillLink

A Freelance Marketplace Database System

Implemented in Oracle XE via DBeaver

Course Title: Database Management System
Course Code: CSC 220
Instructor: Ms. Mehwish Pervaiz
Class: BSIT 5A
Submitted: April 30, 2026

Group Members

Name	Enrollment No.
Zairish Fatima	01-135241-052
Maryam Qadir	01-135241-028
Saim Zafar	01-135241-045

Contents

1	Introduction	2
2	Purpose and Limitations	2
2.1	Problems with Manual or Unstructured Systems	2
2.1.1	Manual System Issues	2
2.1.2	No Tracking	3
2.1.3	Inefficiency	3
2.2	How SkillLink Solves These Problems	3
3	Objectives	3
4	Functional Requirements	4
5	System Overview	5
5.1	Step 1: User Registration	5
5.2	Step 2: Client Posts a Project	5
5.3	Step 3: Freelancer Places a Bid	5
5.4	Step 4: Client Selects a Bid	6
5.5	Step 5: Contract is Created	6
5.6	Step 6: Payment is Made	6
5.7	Step 7: Review is Given	6
5.8	Relationships Overview	6
6	Main Entities	7
7	Key Features	8
8	Scope of the Project	8
8.1	What the system will do:	8
8.2	What the system will not do:	8
9	Expected Outcomes	9
10	Work Division	9
11	Conclusion	10

1 Introduction

SkillLink is a database project that models how a freelancing platform works at the data level. The system is based on a simple real-world scenario where clients post work and freelancers complete it. SkillLink is the platform that connects them, keeps track of everything that happens between them, and stores all of it in a structured, reliable way.

Platforms like Upwork or Fiverr work exactly like this in real life. Someone posts a job, skilled workers apply, a deal is made, the work gets done, payment happens, and both sides leave a review. SkillLink covers that whole process as a database. There is no website or app involved. The focus is on the tables, the relationships, the constraints, and the logic behind the scenes.

We picked this project because it fits almost every topic from our DBMS course into one meaningful system. Specialization, many-to-many relationships, normalization, and complex queries are all part of this design. Every table and every relationship in the database represents something that actually happens when two people do business on a freelancing platform.

This project was chosen because it represents a real-world system that we are already familiar with.

The design uses ER concepts such as specialization, where users are divided into clients and freelancers. Entities such as bids and reviews are closely related to other entities and are linked through foreign keys to maintain data integrity. Many-to-many relationships are resolved using junction tables such as user skills and project category.

The entire project is built and run in Oracle XE using DBeaver as the database client.

2 Purpose and Limitations

2.1 Problems with Manual or Unstructured Systems

Before a structured database system like SkillLink, freelancing interactions are typically managed through informal channels such as email threads, spreadsheets, or messaging apps. These approaches suffer from several critical issues:

2.1.1 Manual System Issues

Without a central database, all records of projects, bids, contracts, and payments must be managed by hand. A client tracking multiple freelancers across different projects has no single reliable source of truth. Data is scattered, duplicated, or simply lost when files are misplaced or communication threads grow too long.

2.1.2 No Tracking

Manual systems provide no way to systematically track the status of a bid, the progress of a contract, or the history of payments. There is no audit trail. If a dispute arises, neither party has a reliable record of what was agreed, what was paid, and when. The absence of structured logging makes accountability nearly impossible.

2.1.3 Inefficiency

Looking up past transactions, finding which freelancers worked on which projects, or calculating total earnings requires manually scanning records. This is time consuming and error prone. There is no way to run a query or generate a report without manually compiling data from multiple sources.

2.2 How SkillLink Solves These Problems

SkillLink addresses every one of these issues by replacing unstructured information with a fully relational database:

- All user registrations, project postings, bids, contracts, payments, and reviews are stored in clearly defined tables with enforced constraints, eliminating data loss and duplication.
- Every record is timestamped and linked through foreign keys, creating a complete and verifiable audit trail for every interaction on the platform.
- SQL queries replace manual searching. Retrieving the top-rated freelancers, the total earnings per freelancer, or the most active clients takes a single statement rather than hours of manual work.
- Disputes, notifications, and file metadata are stored in dedicated tables so that every aspect of a freelancing transaction is captured and retrievable.

3 Objectives

The goals of this project are:

- Design a complete database for freelance interactions between clients and freelancers.
- Store project and bidding data in a structured and consistent way.
- Track contracts and payment records linked to those contracts.
- Maintain a review and rating system with updated records of freelancer performance.
- Support data retrieval and analysis through meaningful SQL queries.
- Apply complex SQL including joins, aggregation, filtering, and grouping to analyse projects, bids, and user performance.

- Normalise the database up to Third Normal Form (3NF) to reduce redundancy and improve data consistency. The design ensures no repeating groups, no partial dependencies, and no transitive dependencies.

4 Functional Requirements

The following functional requirements define what the SkillLink system must support at the database level. Each requirement maps directly to one or more tables and constraints in the schema.

- **User Registration.** The system must allow individuals to register as either a Client or a Freelancer. Common information such as name, email, password, and phone is stored in the users table. A subtype profile record is created in client profile or freelancer profile depending on the user role. Each email address must be unique across the platform.
- **Project Posting.** Registered clients must be able to post projects. Each project stores a title, description, budget, deadline, status, and a creation timestamp. The status field is constrained to the values open, in progress, completed, and cancelled. A project can be assigned to one or more categories through the project category junction table.
- **Bidding.** Freelancers must be able to submit bids on open projects. Each bid records the proposed amount, estimated delivery time, and submission timestamp. A freelancer may submit only one bid per project, enforced through a UNIQUE constraint on the combination of project id and freelancer id. Bid status is constrained to pending, accepted, or rejected.
- **Contract Creation.** When a client selects a bid, the system must create a contract linked to that winning bid. The contract stores the agreed amount, start date, end date, and status. The client, freelancer, and project details are all reachable through the bid reference, keeping the design free of redundant foreign keys. All other bids on the same project are updated to rejected.
- **Payment Tracking.** The system must record all payments made against a contract. Each payment stores the amount, payment date, method, and status. Multiple payments can be linked to one contract to support milestone-based projects. Payment method is constrained to bank, easypaisa, jazzcash, or card.
- **Reviews and Ratings.** After a contract is completed, both the client and the freelancer must be able to submit a review. Each review stores a numeric rating from 1 to 5, a written comment, and a creation timestamp. Reviews are linked to a contract, a reviewer, and a reviewee, all enforced through foreign keys. Each user can submit only one review per contract.
- **Skill and Category Management.** Freelancers must be able to list their skills through the user skills junction table. Projects must be assignable to one or more

categories through project category. These associations support matching between freelancer capabilities and project requirements.

- **Dispute Handling.** Either party must be able to raise a dispute against an active contract. Each dispute records the raising user, a reason, the current status, and a creation timestamp. Status is constrained to open, resolved, or rejected.
- **Notification Storage.** The system must store notifications sent to users. Each notification records the recipient, message content, date, read status, and type such as bid, payment, contract, or review.

5 System Overview

This section explains how the platform works from start to finish. Understanding this flow is important because the database design follows directly from it. Every step below maps to one or more tables and operations in Oracle SQL.

5.1 Step 1: User Registration

A person signs up and is registered as either a Client or a Freelancer. Both types share basic information like name and email, which goes into the users table. After that, a separate profile record is created. Clients get a record in client profile and freelancers get one in freelancer profile. This separation is based on the concept of specialization in database design, where one general entity splits into more specific types.

5.2 Step 2: Client Posts a Project

Once registered, a client can post a project. The project includes a title, a short description, a budget, a deadline, and a category. It is stored in the projects table with a status of open, which means freelancers can now see it and submit bids.

5.3 Step 3: Freelancer Places a Bid

Any freelancer can look at open projects and submit a bid. A bid is essentially a proposal that says: I can do this work for this price and deliver it in this many days. The bid goes into the bids table, linked to both the project and the freelancer. Its status starts as pending. Each freelancer can only submit one bid per project, and this is managed through database constraints.

5.4 Step 4: Client Selects a Bid

The client reviews all bids on their project and picks the one they like. The chosen bid is marked as accepted and all other bids on that project are updated to rejected. A new record is then added to the contracts table and the project status changes from open to in progress. Each contract is created from one accepted bid, ensuring a one-to-one relationship between a contract and the selected bid.

5.5 Step 5: Contract is Created

The contract is the formal agreement between the client and the freelancer. It is stored in contracts and is linked to the selected bid. The client, freelancer, and project details are accessible through the bid reference. It also records the agreed amount and the current status of the contract.

5.6 Step 6: Payment is Made

When the work is done or a milestone is reached, the client releases payment. Payments are made by the client associated with the contract. This is recorded in the payments table and linked to the contract. A contract can have more than one payment record in cases where the project is split into multiple milestones.

5.7 Step 7: Review is Given

After the contract is completed, both the client and the freelancer can leave a review. Each review includes a rating from 1 to 5 and a written comment. Reviews are stored in the reviews table. The system will maintain updated records of ratings and performance based on the reviews submitted.

5.8 Relationships Overview

The following summarises how all the main entities connect to each other.

- One client can post multiple projects, but each project belongs to only one client (1:M).
- One project can receive multiple bids, while each bid is linked to one project (1:M).
- Freelancers can bid on multiple projects, creating a many-to-many relationship between freelancers and projects through bids (M:N).
- Each accepted bid results in one contract, ensuring a one-to-one relationship between a contract and the selected bid.
- One contract can have multiple payments in case of milestone-based work (1:M).
- Reviews are linked to contracts and users after project completion (1:M).
- Each user can submit at most one review per contract.

6 Main Entities

- **Users** represents all individuals using the system, including both clients and freelancers. It stores shared information such as name, email, password, phone, and created at. Users is the supertype and is specialised into two subtypes. Client stores company name and location. Freelancer stores bio, hourly rate, rating, experience level, and availability. This is a disjoint and total specialisation, meaning every user must be either a Client or a Freelancer, never both.
- **Projects** contains details of work posted by clients, including title, description, budget, deadline, and status.
- **Bids** represents proposals submitted by freelancers for projects. Each bid is uniquely identified and is associated with a specific project and freelancer through foreign keys. It stores amount, delivery time, status, and submitted at.
- **Contracts** is created when a bid is accepted. It stores the winning bid through bid id and records the agreed amount, status, start date, and end date. The client, freelancer, and project details are all reachable through the bid reference, keeping the design free of redundant foreign keys.
- **Payments** stores transaction details tied to a contract, including amount, pay date, method, and status. Multiple payments can exist per contract for milestone-based work.
- **Reviews** allows users to provide a rating from 1 to 5 and a written comment after a contract is completed. Each review is linked to a contract, a reviewer, and a reviewee using foreign keys. It stores reviewer id, reviewee id, rating, comment, and created at.
- **Skills** stores the different skills that freelancers can have, such as web development, graphic design, or data entry, along with a category to group similar skills together.
- **Categories** is used to classify projects into different domains or types, such as IT, writing, or marketing. Each category has a name and a description.
- **User Skills** is a junction table that resolves the many-to-many relationship between users and skills.
- **Project Category** is a junction table that resolves the many-to-many relationship between projects and categories.
- **Project Files** stores metadata of files attached to a project, including file name, file type, and upload date.
- **Disputes** tracks disputes raised during a contract. It stores the raised by user, a reason, status, and created at.
- **Notifications** stores system notifications sent to users, including message, date, is read, and type, so the system knows what kind of notification was sent.

In practice, each entity will be uniquely identified using a primary key (e.g., user id, project id, bid id). Relationships between entities will be maintained using foreign keys

(e.g., project id in bids, contract id in payments). Constraints such as UNIQUE, NOT NULL, and CHECK will be used to ensure data consistency.

7 Key Features

- **Bidding System:** Freelancers can submit bids on open projects. Each freelancer can only place one bid per project, managed through database constraints.
- **Contract Management:** When a bid is selected, a contract record is created linked to the winning bid. The client, freelancer, and project are all reachable through the bid reference.
- **Payment Tracking:** Every payment is recorded and linked to a contract. Multiple payments can exist per contract for milestone-based projects.
- **Rating System:** After a contract is done, both parties can rate each other. The system will maintain updated records of freelancer ratings and performance.
- **Notification System:** The system stores notifications for users such as when a bid is accepted or a payment is recorded.
- **Dispute Handling:** If either party has a problem with a contract, a dispute can be raised and tracked through the disputes table.
- **Skill and Category Matching:** Freelancers can list their skills and projects can be assigned to categories, making it easier to find the right match.
- **SQL-Based Analysis:** The system supports complex SQL queries including joins, aggregation, filtering, and grouping to analyse projects, bids, and user performance.

8 Scope of the Project

8.1 What the system will do:

- Store all data related to users, projects, bids, contracts, payments, and reviews.
- Enforce data integrity through primary keys, foreign keys, check constraints, and unique constraints.
- Support complex data retrieval and analysis through SQL queries using joins, aggregation, filtering, and grouping.
- Analyse freelancer performance, project activity, and payment history through query results.
- Demonstrate proper normalisation up to 3NF.

8.2 What the system will not do:

- It will not include a real-time payment gateway or financial processing.

- It will not have a front-end interface, website, or mobile application.
- It will not store actual files. The project files table stores only metadata like file name and upload date.
- It will not handle real-time notifications. The notifications table is for data storage only.

9 Expected Outcomes

By the end of this project, the group will have produced:

- A complete ER diagram showing all entities, relationships, and ER concepts used.
- A full Relational Data Model with all tables, columns, and constraints defined.
- Working Oracle SQL scripts that create all tables and insert sample data, executed in DBserver on Oracle XE.
- SQL queries demonstrating joins, grouping, subqueries, and aggregate functions.
- Insights into freelancer performance, platform activity, and transaction history through query results.
- A structured report documenting every design decision and normalisation step.

10 Work Division

The project is split into three modules, one per group member. Each member is responsible for designing, implementing, and documenting their assigned part. This makes the work straightforward because each person has full knowledge of what they built.

Table 1: Work Distribution

Member	Module	Responsibilities
Zairish Fatima	Skills, Categories and Notifications	Tables: skills, user skills, categories, project category, notifications. Sample data, join queries and select queries with grouping.
Maryam Qadir	Users, Profiles and Projects	Tables: users, client profile, freelancer profile, projects, project files. Sample data, join queries and specialization implementation.
Saim Zafar	Bids, Contracts, Payments and Reviews	Tables: bids, contracts, payments, reviews, disputes. Sample data, complex queries using aggregation, grouping and multi-table joins.

All three members will work together on the ER diagram and the final documentation.

11 Conclusion

SkillLink is a practical and well-scoped project that goes beyond basic data storage. The platform models a real workflow that people use every day on freelancing sites. The design choices are not random every table exists because something real needs to be stored, and every relationship exists because two things in the real world are actually connected.

From a DBMS perspective, the database demonstrates conceptual design through ER, logical design through normalisation, and physical implementation through Oracle SQL. Complex enough to be meaningful, focused enough to finish properly within the semester.

The goal is not just to pass the project requirement. The goal is to build something that actually makes sense as a database, and that is what SkillLink is designed to be.